

Эмулятор командной строки UNIX-подобной ОС

Выполнила: Серафима Шеверда Сергеевна **Группа** ИКБО-14-25 **Вариант:** №23

Описание проекта

Данный проект представляет собой эмулятор командной строки, работающий в режиме REPL (Read-Eval-Print Loop). Эмулятор симулирует работу в терминале UNIX-подобной операционной системы. Логика файловой системы полностью изолирована и загружается из виртуальной модели в формате JSON.

Описание функций команд эмулятора

В проекте реализованы следующие встроенные функции для обработки команд:

- `ls` — вывод содержимого текущей директории VFS.
- `cd <путь>` — смена текущей директории виртуальной файловой системы.
- `history` — вывод списка всех успешно введенных команд за текущую сессию (Индивидуальное требование).
- `tree` — рекурсивный вывод структуры каталогов в виде дерева с дефисами (Индивидуальное требование).
- `head <файл>` — вывод первых 10 строк содержимого виртуального текстового файла (Индивидуальное требование).
- `exit` — штатное завершение работы эмулятора.

Требования к окружению

- Python версии 3.10 или выше.
- Операционная система: Windows / macOS / Linux.

Для запуска эмулятора необходимо передать обязательные аргументы командной строки: путь к JSON-файлу виртуальной файловой системы (`--vfs`) и путь к стартовому скрипту (`--script`).

```
python src/main.py --vfs src/vfs.json --script src/start.sh
```

Пример работы эмулятора:

```
Эмулятор командной строки [23 Вариант]
[DEBUG] модель VFS успешно загружена из: src/vfs.json
Введите "exit" для выхода.

--- Выполнение стартового скрипта: src/start.sh ---
[vfs /]> tree
--- home
    --- user
```

```
        --- docs
        --- readme.txt
--- var
    --- log
[vfs /]> cd home
[vfs /home]> cd user
[vfs /home/user]> head readme.txt
Строка 1
Строка 2
Строка 3
Строка 4
Строка 5
Строка 6
Строка 7
Строка 8
Строка 9
Строка 10
[vfs /home/user]> history
1 tree
2 cd home
3 cd user
4 head readme.txt
5 history
--- Serafima Script успешно выполнен ---

[vfs /home/user]>
```

Запуск автоматических тестов

Для проверки работоспособности парсера команд и обработки комментариев используются юнит-тесты:

```
python -m unittest discover -s tests
```